

Lab 5 - SYCL Basics

Advanced Computer Architectures

University of Minho

The Lab 5 focus on the development of parallel code using SYCL by covering some of its advanced features that have a relevant impact on performance. Use a Deucalion node with NVidia GPUs (by specifying the partition `normal-gpu` when submitting a job) only for the exercises that require them. Compile the code either in the frontend or on the compute nodes.

This lab tutorial includes four exercises to be solved during the lab class. Copy the source files, as usual, from `/projects/F202500010HPCVLABUMINHO/ACA/lab5` to your home directory in Deucalion:

```
cp -r /projects/F202500010HPCVLABUMINHO/ACA/lab5 .
```

To load the compiler in the environment use the following commands (do not forget to also use them in your submission script):

GNU Compiler: `module load GCC/12.3.0.`

CUDA Environment: `module load CUDA/12.4.0.`

OneAPI Environment: `source /share/apps-x86/ohpc/pub/apps/intel/oneapi/setvars.sh.`

You can use other GNU/CUDA combinations at your own risk, but OneAPI is always required to use SYCL. Use either `nvprof` or `nsys` command line tools to profile GPU code if necessary.

To compile the code using at least the following options for the Intel compiler:
`icpx -fsycl -fsycl-targets=nvptx64-nvidia-cuda ...`

1 Single Device Compute

Goals: to develop skills in developing code with SYCL for CPU or GPU execution.

Consider the classic square matrix-matrix multiplication as case study to explore SYCL functionalities throughout the following exercises. The provided code skeletons already contain an implementation of this algorithm, as well as the required matrix initialisations.

Lab 1.1 Complement the CPU skeleton with the SYCL code required to parallelise the matrix multiplication only using CPUs. You will require to define a SYCL queue to submit the workload to, a buffer for each matrix (with the respective accessor), and a parallel for (that can be define as two-dimensional). See the comments on the code for tips on where to code what.

Measure the execution time and compare it to the sequential implementation. Does the speedup match expectations?

Lab 1.2 Do the same for the GPU-only code. See the comments on the code for tips on where to code what. How many GPUs are used by default? Is it possible to modify the code to use more/less?

Measure the execution time and compare it to the CPU implementation. Does the speedup match expectations?

2 CPU-GPU Compute

Goals: to develop skills in developing code with SYCL for simultaneous CPU-GPU execution.

Lab 2.1 Parallelise the code for simultaneous execution on CPU and GPU using a unified memory space. You will need to allocate the matrices using SYCL's `malloc_shared`, but their initialisation can use the same approach as in `malloc_shared`. Each device must compute a portion of the matrices, which have to be managed by the developer. One possible strategy is to distribute half of the rows to each device, but it is not the most efficient.

Measure and compare the performance with the sequential code. How much faster is the implementation using SYCL?

Lab 2.2 Assume that you need to process several 32 batches of matrix-matrix multiplications. Complete the respective skeleton to process this workload using all of the devices inside the compute node.

Measure and compare the performance with the sequential code for the same workload. How much faster is the implementation using SYCL?